



BSc (Hons) in **Information Technology**  
**Specialized in AI**

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 01

**Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

### Part 1: Code Analysis & PEAS Evaluation

#### Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

#### Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure  
Environment  
Actuators  
Sensors



2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
elf.width  
self.height  
self.walls  
self.food_positions  
self.opponents
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi agent :self.opponents adds other independent entities that move and interact with the agent each turn, and their behavior affects the agent's score.

## Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of logs (percepts) an agent has received by its sensors up to a certain point

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

S - sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially observable : the percept never actually tells you where the food is. You just get `smells_food` (true/false for your current cell) and `remaining_food` (a count), so you know food exists but not its location. Since the agent can't fully reconstruct the true state of the grid from what it perceives, it's not fully observable.

### Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

P : Performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Because the performance measure is supposed to judge the agent based on how well it actually does in the environment, not how much "work" it did internally to decide.

If the penalty was based on internal effort (like how many steps the agent's logic took to decide), an agent could theoretically compute a lot and still crash into walls all the time — and still get a good score. That doesn't actually tell us if the agent is any good.

By tying the -5 penalty to `tuple(new_pos)` in `self.walls`, it's checking something real and observable in the environment (did the agent actually hit a wall or not), so:

it's objective : anyone can verify it just by looking at the environment state  
it works no matter what agent design is plugged in (random, rule-based, etc.)  
the agent can't "cheat" the score by just doing more internal processing without improving actual behaviour

Basically, performance should be measured by outcomes in the world, not effort inside the agent's head — that's the whole point of separating the agent from the environment in PEAS.

## Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

### Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

By adding `self.toxic_traps` to the environment but not exposing it through `get_percept()`, the environment changes from fully observable to partially observable.

An environment is fully observable when the agent's sensors give it access to the complete state relevant to its decisions at every time step. Before adding traps, the agent could "see" everything that mattered (its position, walls, food, opponents). Now, `toxic_traps` is part of the true environment state and it does affect the agent (it causes a score penalty in `execute_action`), but the agent has no sensor data telling it where traps are. So there's a gap between the real state of the world and what the agent can perceive — which is exactly the definition of partial observability. The agent is forced to act (and potentially get penalized) based on incomplete information, since it can't directly perceive a feature of the environment that actually impacts its outcome.

### Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing

expected utility).

mells\_toxin reduces the agent's uncertainty about the environment state, and a rational agent uses that reduced uncertainty to pick actions that avoid a known penalty moving its behavior closer to expected-utility-maximizing rather than reactive/uninformed.

### Step 2.3: Modifying Action Execution & Visual Rendering (execute\_action & draw\_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic "Vacuum World Exploit" is where the performance measure rewards the agent for the amount of dirt cleaned up rather than for how clean the floor actually ends up being. A "rational" agent chasing that flawed metric can rack up unbounded score by repeatedly sucking up dirt and then dumping it back onto the floor, just to clean it up again and again — gaming the metric without ever achieving the real goal of a clean floor. It's the textbook lesson that a performance measure should be based on the desired state of the environment, not on the agent's actions, or the agent will find a degenerate way to exploit it. That's exactly why the trap penalty here has to hit `self.score` directly for the undesirable state (standing on a trap), rather than something the agent could grind for points

Finalize and Lock Lab 01 Analytical Evaluation

**Lab 01 code analysis and theoretical evaluation complete and verified!**



**FACULTY OF COMPUTING**